

Diseño de Sistemas de Información – K3001/K3101

Trabajo Práctico Anual – SEGUNDA Entrega

Justificaciones a las decisiones de diseño

Integrantes del grupo

- Lucas Pagnaro
- Juan Cruz Castelo
- Cynthia Abbate
- Paula González
- Bruno de Angelis

Ayudantes designados

- Gonzalo Turri
- Thomas Luca

Grupo 9
27-5-2025

Tabla de contenido

Fuente Dinámica 2

 Repositories..... 2

 Domain..... 2

Fuente Proxy 3

 Domain..... 3

 Services..... 3

 DTOS 3

Servicio Agregador 4

 Domain..... 4

 DTOS 4

 Repositories..... 4

Fuente Estática 6

 Justificaciones en la entrega anterior 6

Fuente Dinámica

Repositories

Implementamos interfaces por principios **SOLID** (ISP y DIP), una interfaz expone solo lo necesario, sin depender de implementaciones concretas.

ID como Long

Se optó por modelar las ID/handlers de tipo Long para facilitar la persistencia en esta entrega, considerando que en futuras iteraciones se planea implementar una base de datos de la cual obtendríamos un ID en un formato específico.

Id en hecho y solicitud modificación para que sea más sencilla su identificación a lo largo del sistema. Además la utilizamos para filtrar y buscar (un hecho, una solicitud) según su ID

Services

Utilizan interfaces para minimizar acoplamiento ya que estas exponen solo lo necesario

Domain

Solicitud de Modificación:

-Se conserva el hecho original en un historial (historialModificados) para evitar pérdida de información y permitir trazabilidad de cambios y el control del flujo de decisiones.

Contribuyente:

-Se decidió agregar una flag que permite sabersi el usuario es anónimo o registrado, lo que permite verificar si un hecho puede ser o no modificado.

Hecho:

-Teniendo en cuenta el Single Responsibility Principle, implementamos en la clase hecho la responsabilidad de saber si éste puede ser editado o no.

Fuente Proxy

Domain

Como entidades de dominio tenemos la clase estrategiaProxy que implementa la interfaz estrategia. De esta manera podemos tener una estrategia con los datos necesarios para obtener los hechos de una fuente ya sea única o de múltiples instancias.

Buscamos ser específicos con las interfaces acorde al principio Solid de Interface Segregation Service para garantizar muchas interfaces con comportamientos bien definidos y responsabilidades limitadas.

Con respecto a las cualidades de diseño buscamos extensibilidad con las estrategias de importación; y garantizamos mantenibilidad con el modelado en capas.

Services

Optamos por un service locator para administrar todas las posibles implementaciones de otras APIs que provean hechos. De esta manera garantizamos código limpio, cohesivo, extensible y mantenible sin necesidad que el controller conozca todos los modos de obtener hechos.

Al no interactuar tanto con la capa de dominio directamente usamos un Seeder para tener datos de prueba y ejemplos de cómo usamos la aplicación. Con la ayuda de postman aportamos un poco de testabilidad pero no fue el fuerte del modulo

DTOS

Los dtos con el nombre del servicio que vamos a consumir son nuestras interfaces entrantes, y los hechos siempre van a salir en formato HechoOutputDTO para que los consuma el servicio de agregación. Esta es nuestra interfaz saliente.

Servicio Agregador

Domain

- Id en colección, hecho, categoría, etiqueta, solicitud de eliminación. Agregamos un ID para que sea más sencilla su identificación a lo largo del sistema. Además la utilizamos para filtrar y buscar (un hecho, una categoría, etiqueta) según su ID
- Fuente como clase abstracta. La heredan los 3 tipos de fuentes con el atributo tipo. Tienen más sentido como data class, ya no presentan comportamiento.
- Método en solicitud de eliminación “RechazarPorSpam()” para que funcione automáticamente.

DTOS

- De input y output tenemos colección, hecho, solicitud de eliminación y estado de solicitud.
- En hechos referenciamos las etiquetas y categorías con sus ID para no embeber el objeto entero e ir a buscarlo a su respectivo repositorio.
- Fuente tiene su DTO con el archivo, tipo y proxy.
- Criterios: fueron divididos en varios DTOS para tratarlos polimórficamente y que cada uno serialice distinto dependiendo el criterio. Esto favorece la extensibilidad: si se necesita agregar un nuevo tipo de criterio, solo se debe crear una nueva subclase.

Repositories

Implementamos interfaces por principios **SOLID** (ISP y DIP), una interfaz expone solo lo necesario, sin depender de implementaciones concretas.

ID como Long

Se optó por modelar las ID/handlers de tipo Long para facilitar la persistencia en esta entrega, considerando que en futuras iteraciones se planea implementar una base de datos de la cual obtendríamos un ID en un formato específico.

Services

Utilizan interfaces para minimizar acoplamiento ya que estas exponen solo lo necesario

Builder

Implementamos un patron builder para instanciar los hechos combinado con un patron strategy que se encarga de detectar de que fuente proviene el hecho para construirlo de una determinada forma. Si viene de una fuente dinámica se construye con contribuyente, contenido multimedia, atributos que no tendría en un hecho proveniente de una fuente estática.

Detector de Spam LOCAL

Detector local de spam para no depender de servicios externos.

Lo implementamos en la capa de services porque orquesta lógica. no modela comportamiento de objetos. Y si en otra iteración queremos implementar una API no puede estar en el domain.

Refresh colecciones con Estática y Dinámica

La fuente proxy se hace en tiempo real.

Fuente Estática

Justificaciones en la entrega anterior